

programs or code to communicate these errors through an interface that's visible to users.

An easier alternative may be to ensure that these configuration files are well formed. This can be done by using some validator programs that run immediately after installation to confirm that the main programs making up the product will not have any functional issues. These validator programs could then print the errors in a user friendly interface and reduce the need for the main programs to check parameters. In this article, I aim to provide one such example of a validator program that will check that the configuration file is well formed and valid options are provided.

Before you begin on Python

For the purpose of this article, I will assume that you have installed Python 2.7.3. There are good discussion points about choosing Python 2 over Python 3. The main difference is that Python 3 is an improvement over Python 2, but these very improvements have resulted in Python 3 not being backward compatible with some of the already deployed code. Also, not all libraries have been ported to Python 3. These arguments make it simpler to use Python 2, for now.

The development environment used for this article is Windows XP. But the code and the examples should work on any Python 2.7 installation, independent of the OS.

Figure 1 shows the menu items that should be visible if your installation went through fine.

I strongly recommend that you start building the program given below using IDLE (Python GUI) to frequently test and understand if the program's parts are working the way it was envisaged.

IDLE GUI usage is shown like this in this article with the prompt `>>>`.

```
>>> print "using idle gui"
using idle gui
```

The *Python Manuals* are a great starting point for learning Python, especially *The Python Tutorial*.

I would urge you to frequently use these manuals for reference. They also have a very good *Search* capability, if you want to find out something more on a particular topic.

The format of config files

Config files are normally in the following format, organised in terms of sections, with each section having different options. The options are normally organised as name value pairs.

```
[section1]
Option1=Value1
Option2=Value2
[section2]
Option3=Value1
Option4=Value2
```

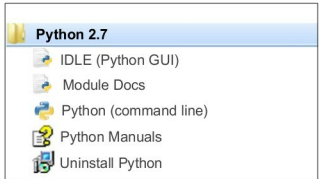


Figure 1: Python 2.7 menu options

An example of a config file could be as follows:

```
#example.cfg
[General]
Customer = CustomerX
[customerx]
BrandingTitle=CustomerX Title
CustomerContact=CustomerX_contact
[customerxy]
BrandingTitle=CustomerY Title
CustomerContact=CustomerY_contact
```

ConfigParser - parsing config files the Python way

Python 2.7 supports a default *ConfigParser* package. Before using the package, you should import it as shown below:

```
>>> import ConfigParser
>>> config=ConfigParser.ConfigParser()
```

After importing it, we create an object of the *ConfigParser* called *config1*. We do this by creating an unnamed object and calling its constructor and assigning it to *config1*.

ConfigParser provides the following interfaces:

- **read(filepath):** This method initialises the *ConfigParser* object with the sections and options in a *.cfg* file, present in the path indicated by *file path*.
- **sections():** This method returns the sections present in the config object. When initialised with a valid config file, this will return a list of section names.
- **options(section):** This method returns the options or keys present in a named section as a list.
- **has_option(section, option):** This method checks whether the section named has the option requested. It returns a 'true' if the option exists and a 'false' if it doesn't.
- **get(section, option):** This method returns the value associated with a particular option or key in the section named.

Let us put together all of the above, using *example.cfg*, which we can assume to be in the following path: *c:\example.cfg*